PSP中間レポート

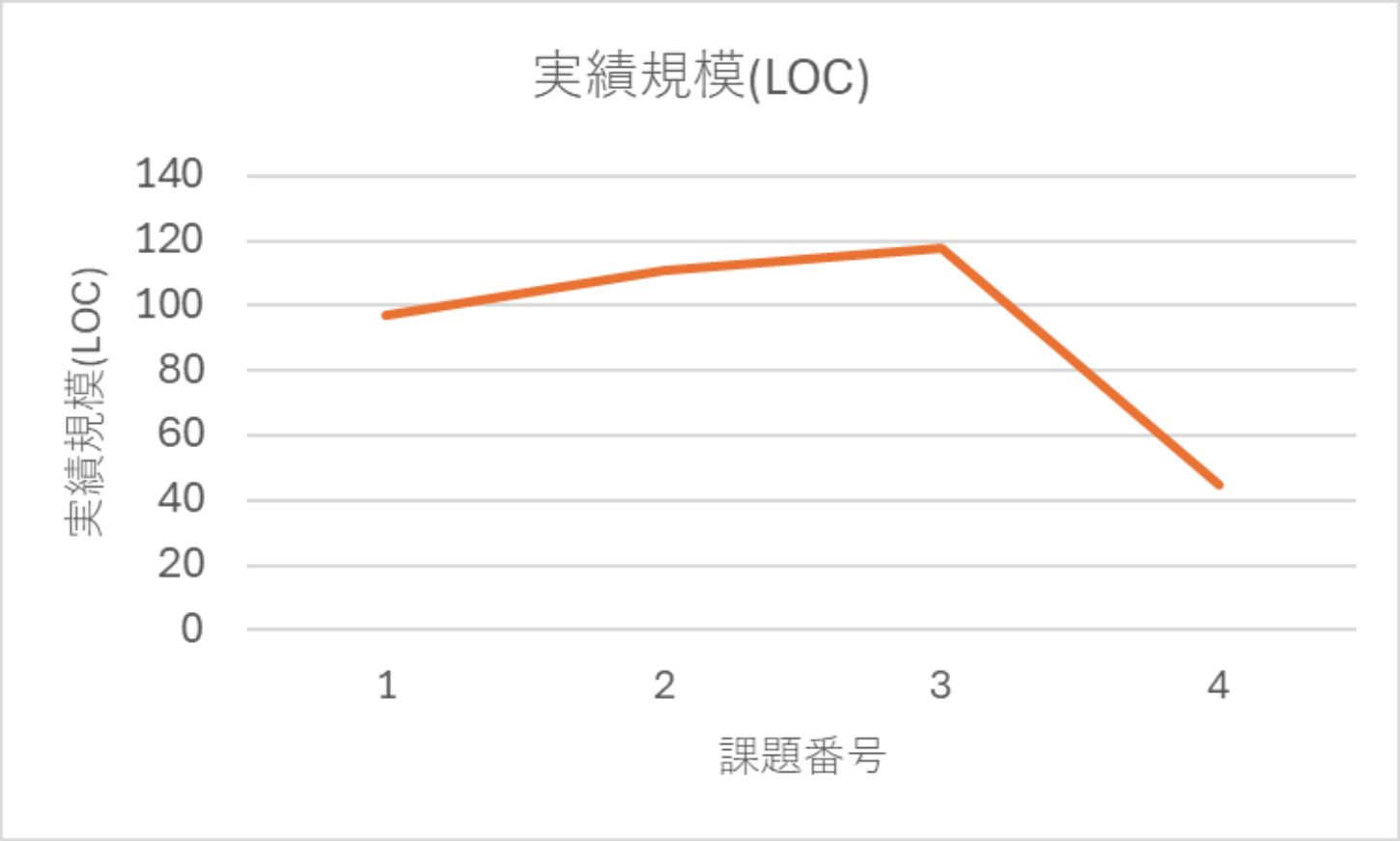
九州工業大学院
情報創成工学専攻 片峯研究室
三留慎太郎

1.1 実績規模(LOC)

課題番号	実績規模(LOC)
1	97
2	111
3	118
4	45

平均:(97+111+118+45)/4=92.75
最大値:118
最小値:45

1.1 実績規模グラフ



課題4はベースコードをかなり利用できたので実績規模が小さくなった

1.2,1.3 規模見積もり誤差(%)

課題番号	計画規模(LOC)	実績規模(LOC)	規模見積もり誤差(%)
1		(97)	
2	70	111	+58.57
3	123	118	-4.07
4	78.3	45	-42.53
合計	271.3	274	+1.00

誤差＝(実績−計画)/計画

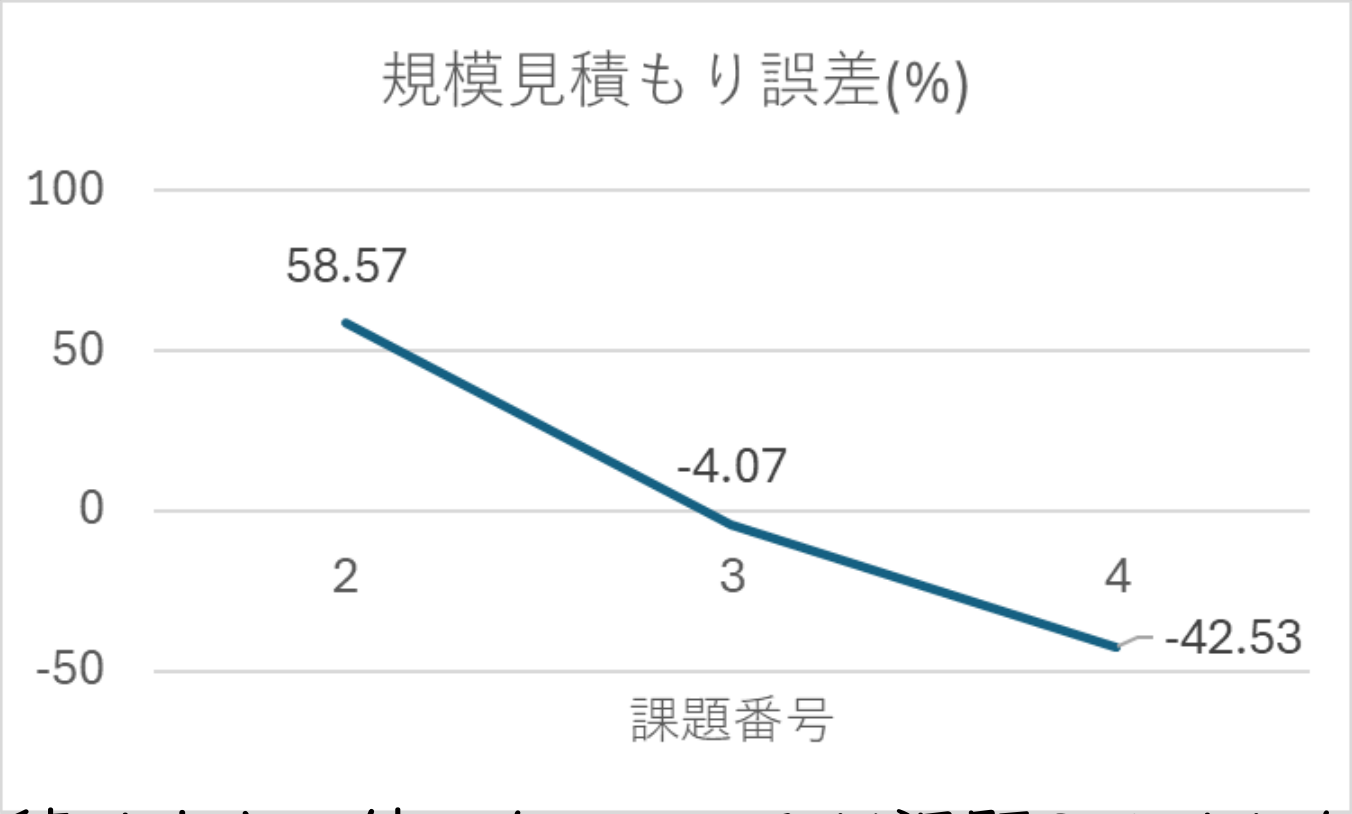
平均(58.57−4.07−42.53)/3=+3.99

最大値:+58.57

最小値:−42.53

累積:+1.00

1.2,1.3 規模見積もり誤差グラフ



平均,累積は小さい値になっているが課題2,4はかなり誤差がある。

規模見積もり誤差の原因と改善

原因：

- 計画段階で実装をイメージできていない
- ベースコードの追加部分の見積もりが雑

改善：

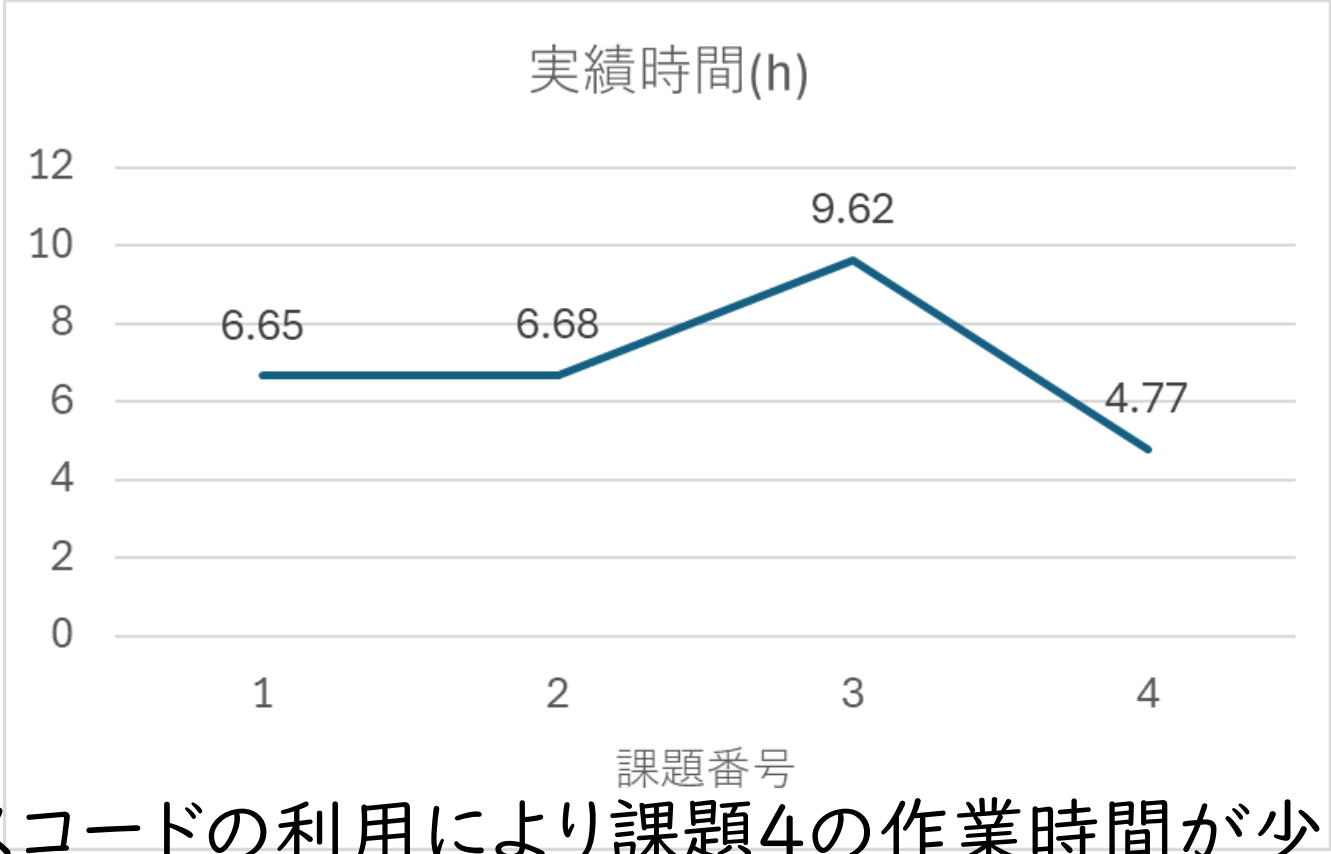
- 実装をイメージして、追加部分に関してもプロキシの規模範囲を利用した根拠のある見積もりを行う

2.1 実績時間(h)

課題番号	実績時間(h)
1	6.65
2	6.68
3	9.62
4	4.77

平均:(6.65+6.68+9.62+4.77)/4=6.93
最大値:9.62
最小値:4.77

2.1 実績時間グラフ



ベースコードの利用により課題4の作業時間が少ない

課題3は1つの欠陥除去に50分かかるなど苦戦

2.2,2.3 時間見積もり誤差(%)

課題番号	計画時間(h)	実績時間(h)	時間見積もり誤差(%)
1	10.87	6.65	-38.82
2	6.1	6.68	+9.51
3	7.42	9.62	+29.65
4	5.57	4.77	-14.36
合計	29.96	27.72	-7.48

誤差＝(実績−計画)/計画

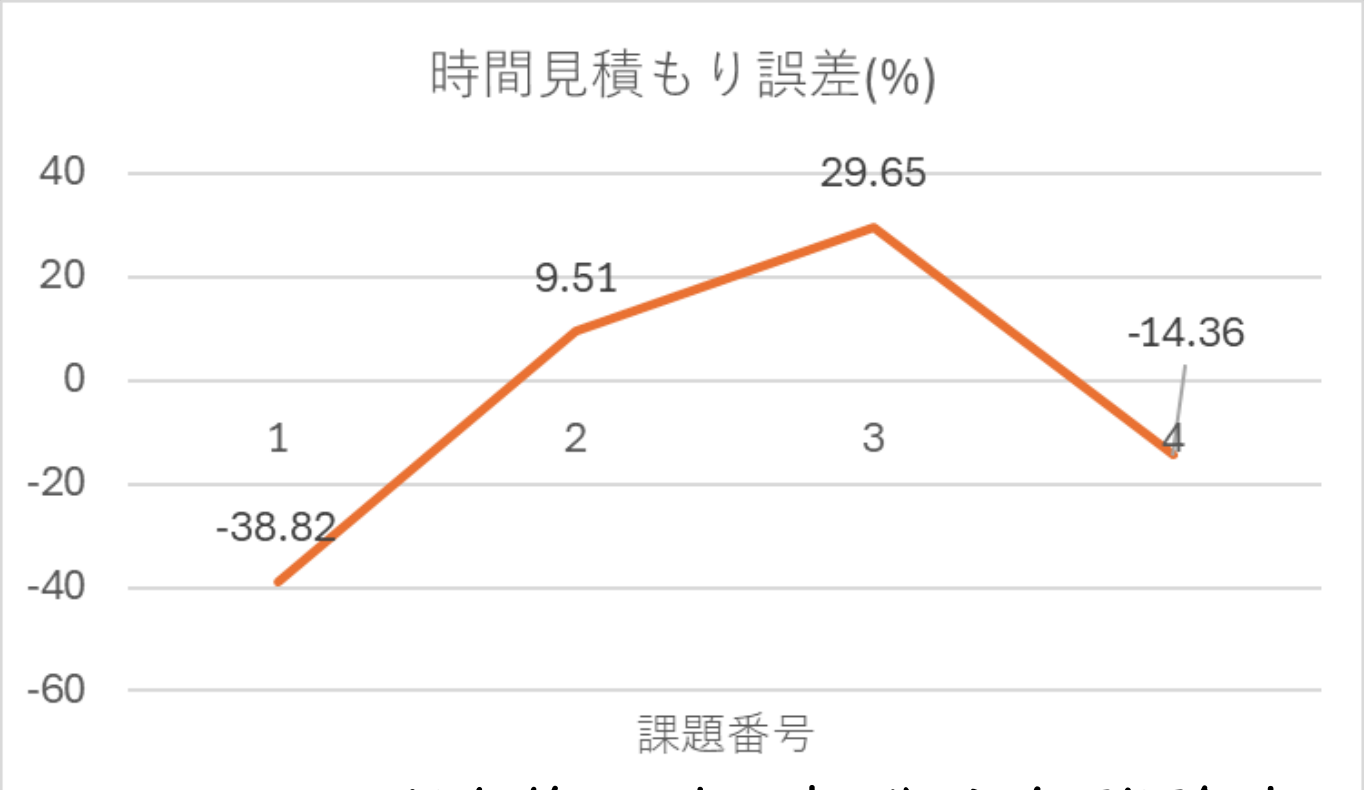
平均(−38.82+9.51+29.65−14.36)/4=−3.51

最大値:+29.65

最小値:−38.82

累積:−7.48

2.2,2.3 時間見積もり誤差グラフ



課題3はベースコードを修正する部分や欠陥除去に苦戦し、時間がかかった。designでの欠陥埋め込みも多かった

時間見積もりの誤差の原因と改善

原因：

- 3ではベースコードを修正し活用することに苦戦し、さらに、designで単体テストに埋め込んだsyntax欠陥の除去に52分かかった。
- 規模見積もりが不正確だった

改善：

- 修正部分に関しては、ベースコードも見直しその修正で利用可能であるかを慎重に判断する
- 新規部品だけでなくベースコード追加部分に関してもプロキシの規模範囲を活用する

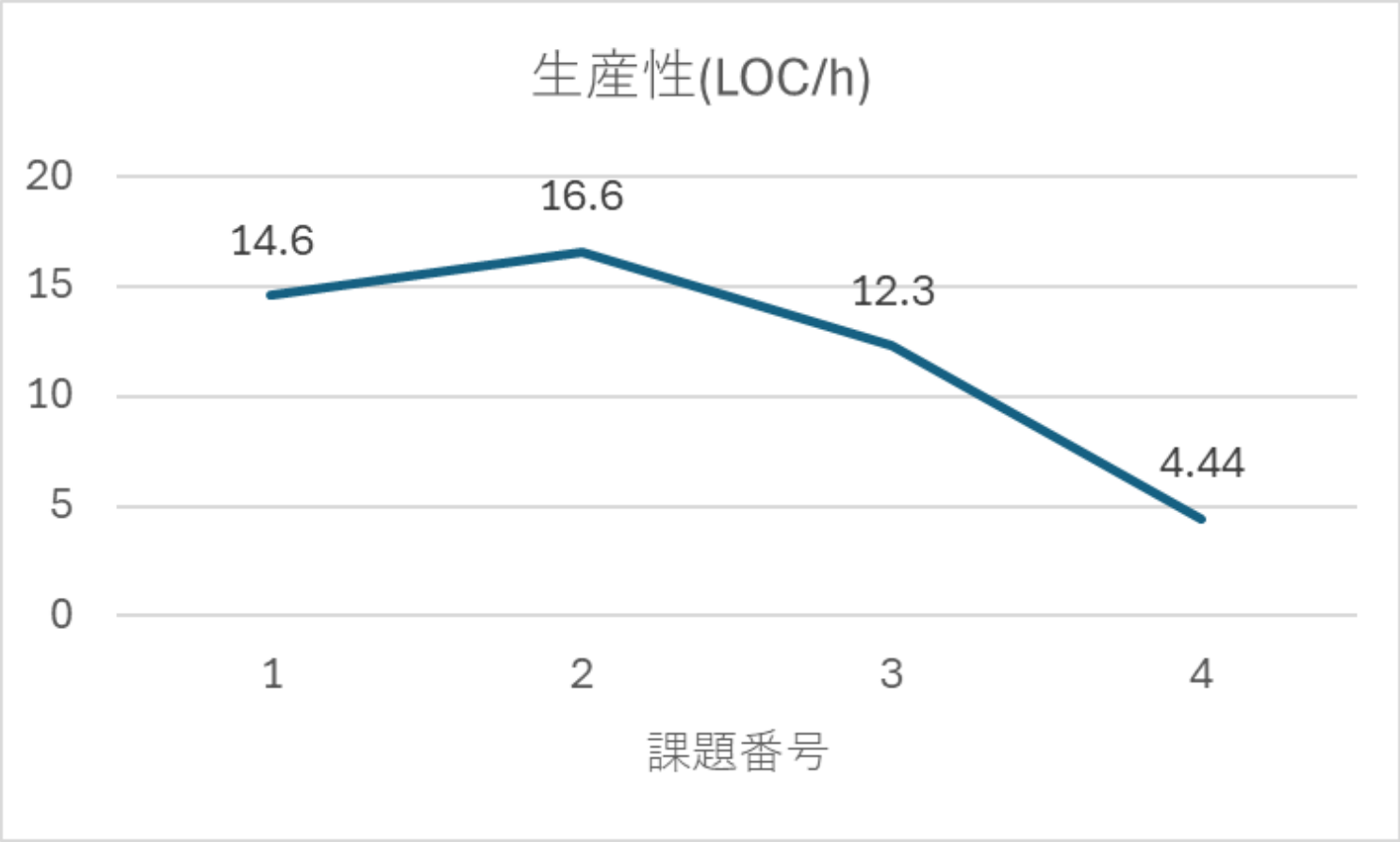
2.4 生産性(LOC/h)

課題番号	実績規模(LOC)	実績時間(h)	生産性(LOC/h)
1	97	6.65	14.6
2	111	6.68	16.6
3	118	9.62	12.3
4	45	4.77	4.44
合計	371	27.72	13.38

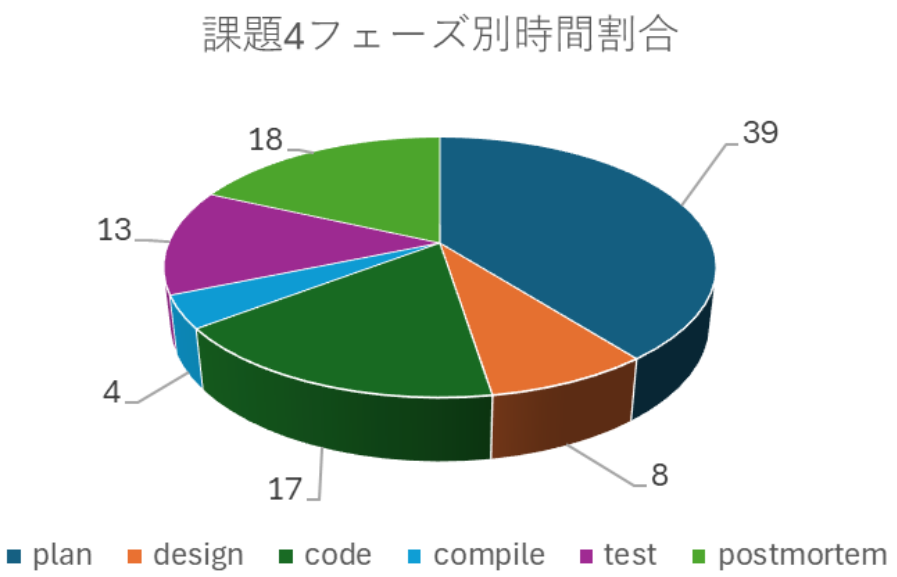
生産性＝実績規模/実績時間

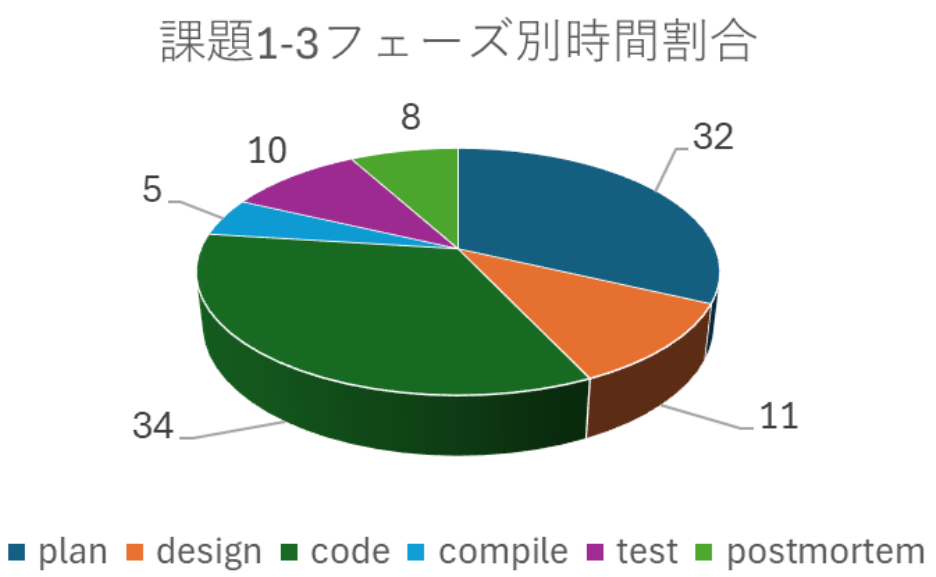
平均：13.38
最大値：16.6
最小値：4.44

2.4 生産性グラフ



生産性に対する考察





課題4に関して、予防的活動の割合がほかの課題に比べ極端に低くなっており、生産性が低くなっていると考えられる

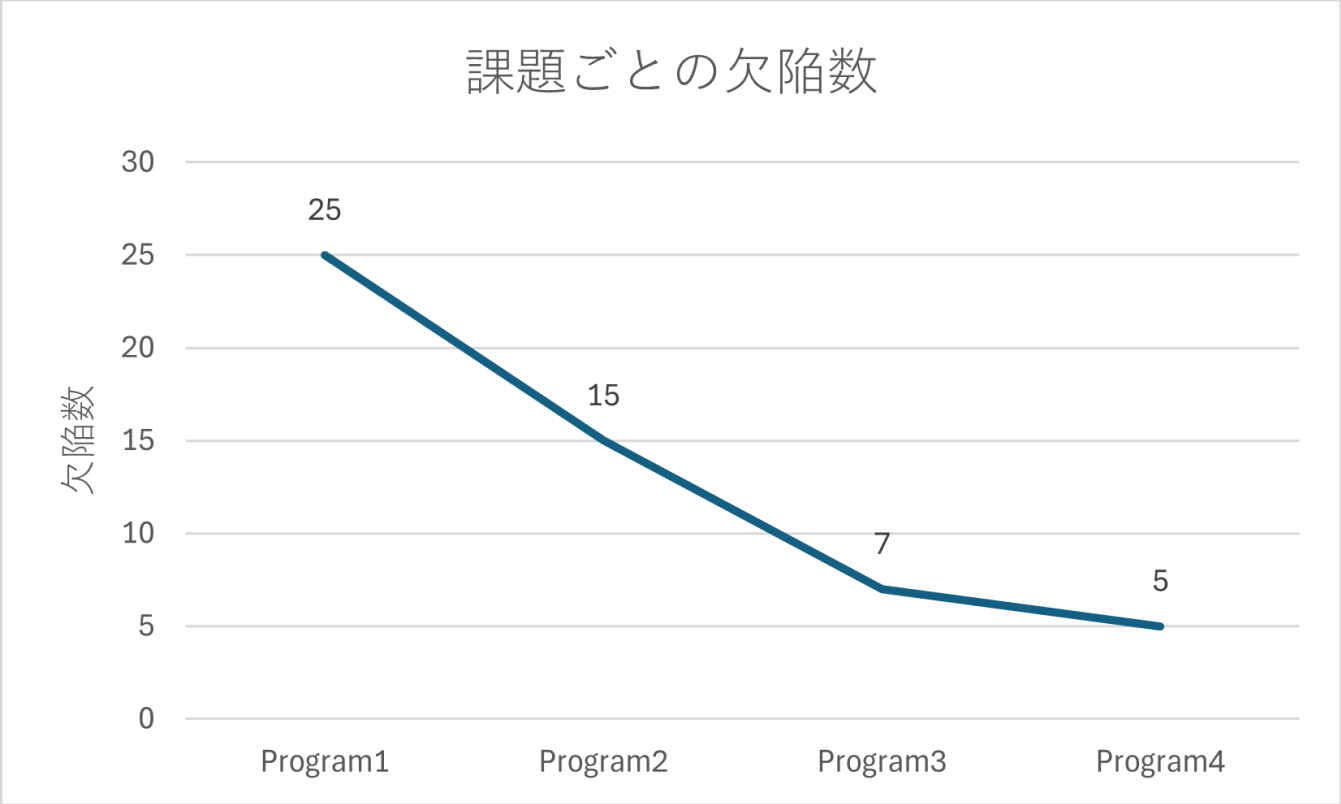
また、規模によってかかる時間が変動しにくい要求仕様書の作成などの作業とPIPの作成に時間がかかりcode以外のフェーズ割合が大きい

3.1 欠陥数(個)

課題番号	欠陥数(個)
1	25
2	15
3	7
4	5

平均:(25+15+7+5)/4=13.0
最大値:25
最小値:5

3.1 欠陥数グラフ



欠陥数に減少傾向がみられる。「public static」「メソッド呼び出しでのクラス指定」などJavaの書き方を思い出してきたことが起因している。

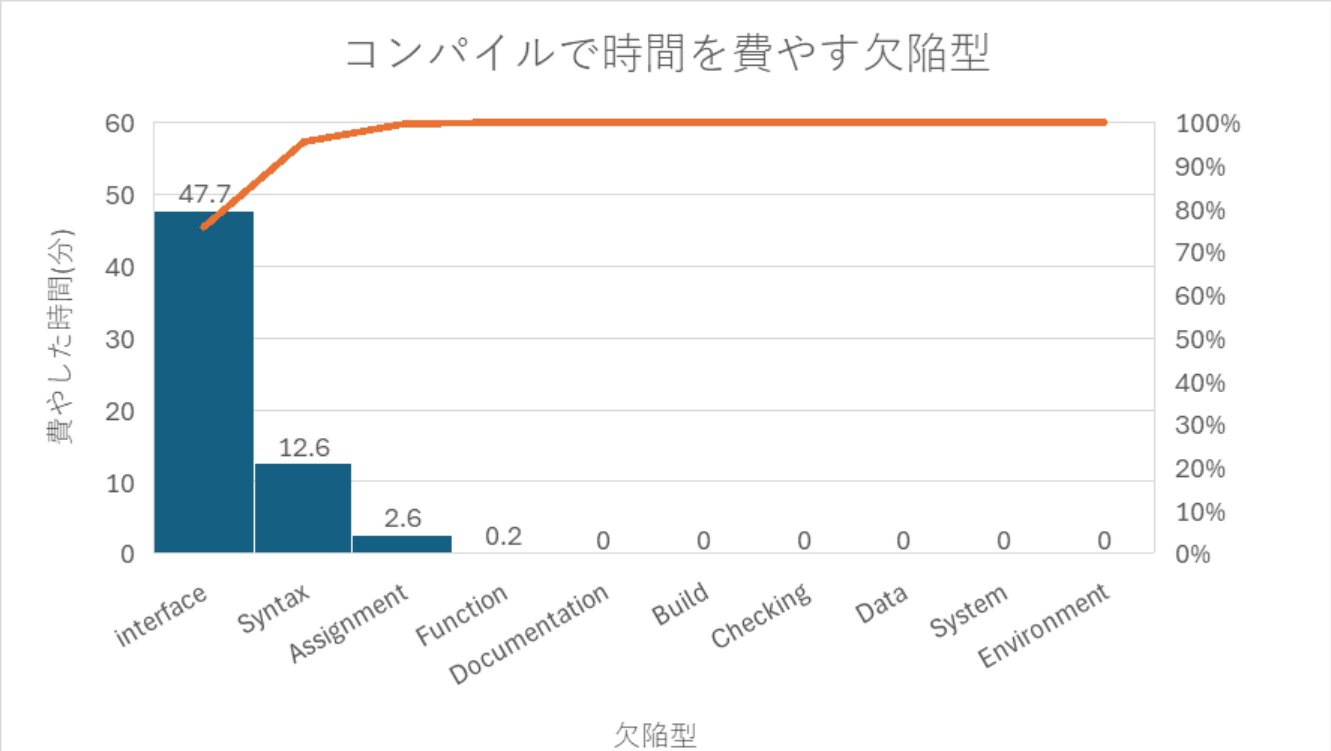
3.2 欠陥密度(個/KLOC)

課題番号	コンパイル欠陥数(個)	コンパイル欠陥密度(個/KLOC)	テスト欠陥数(個)	テスト欠陥密度(個/KLOC)	実績規模(LOC)
1	23	237.0	1	10.3	97
2	2	18.0	9	81.1	111
3	0	0	3	25.4	118
4	2	44.4	3	66.7	45
合計	27	72.8	16	43.1	371

欠陥密度＝(欠陥数/規模)*1000

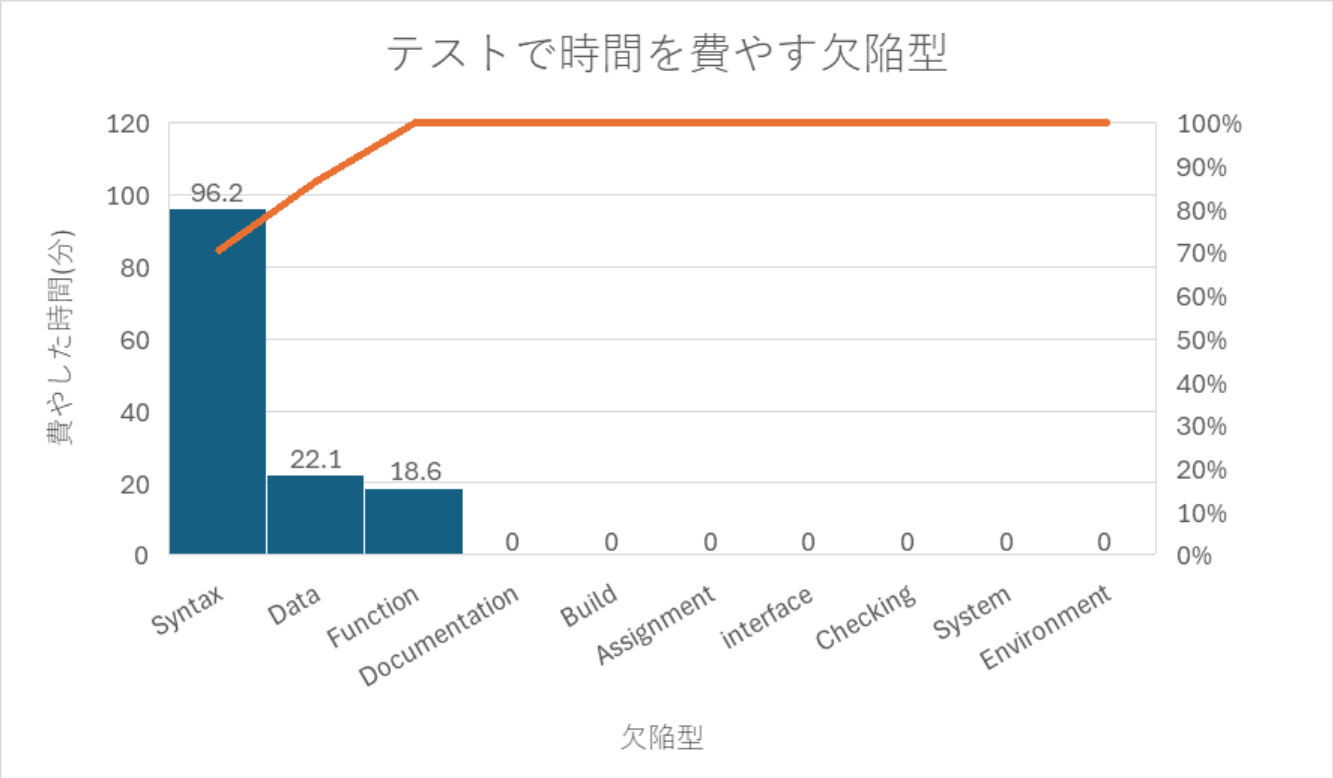
平均,最大値,最小値は表のとおり
テスト欠陥密度が減っていないが、このほとんどはcodeで埋め込まれており、書き間違えが原因であることが多い

3.3 コンパイルで時間を費やす欠陥型



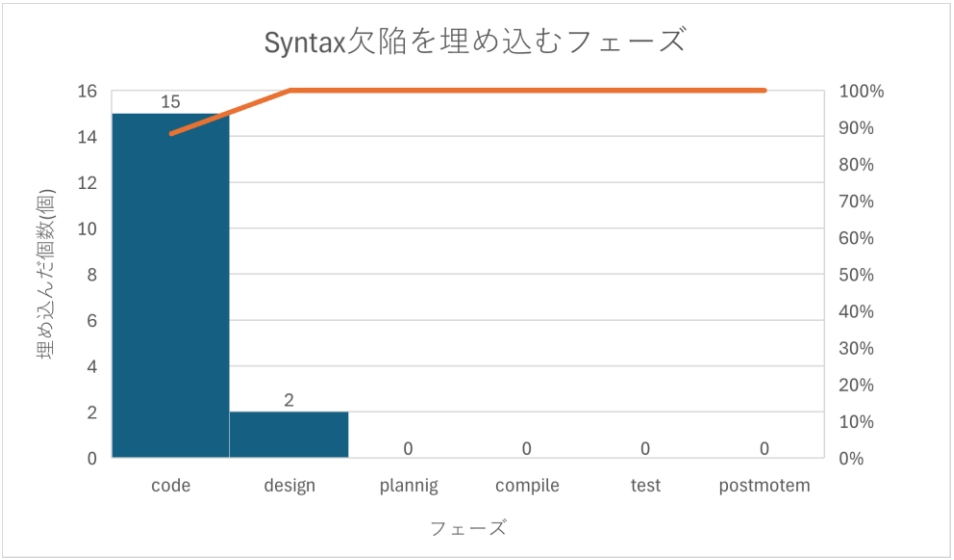
compileで取り除いたinterface欠陥にはメソッド呼び出し元クラス名の指定忘れなどがあったが、課題1では3個44.9分課題2では1個2.8分、課題3,4では0個と減少している

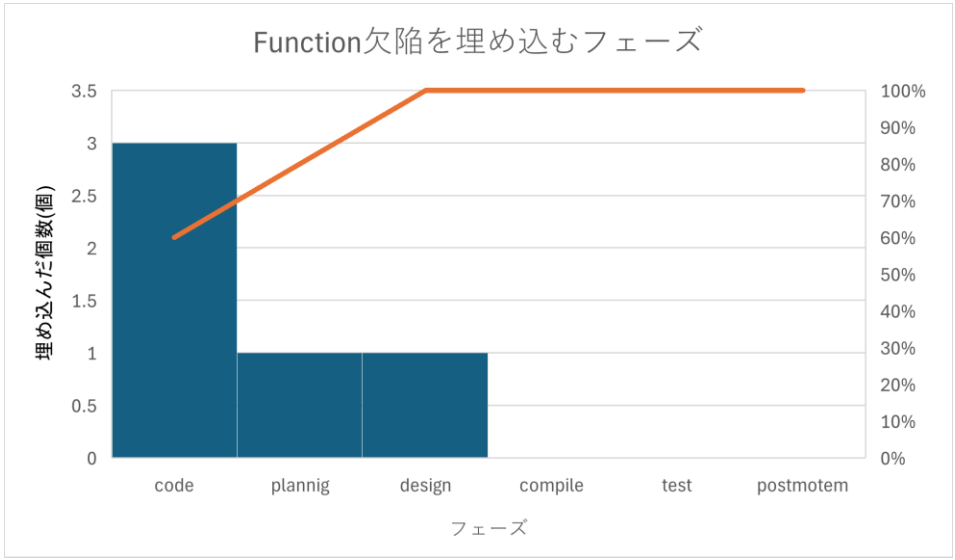
3.3 テストで時間を費やす欠陥型



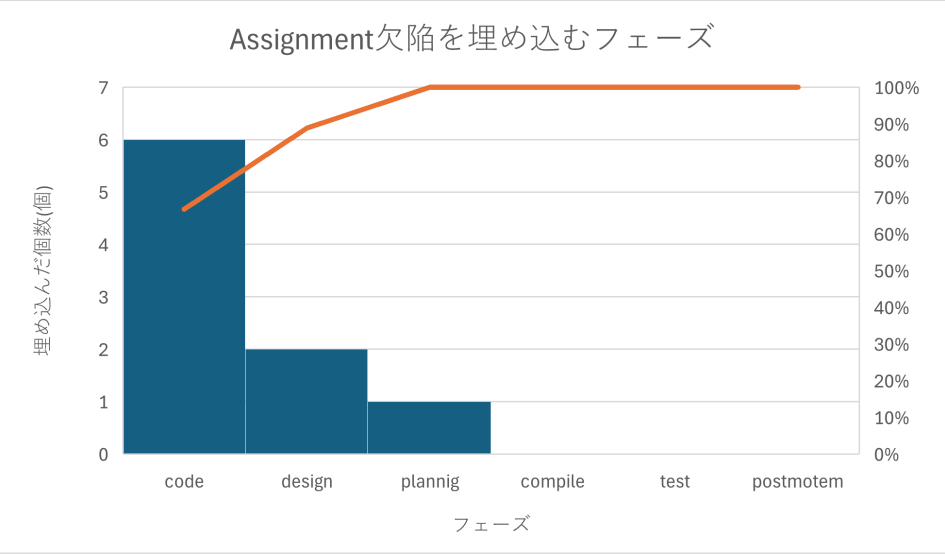
testで取り除いたsyntax欠陥はcodeで埋め込んだ書き間違えがそのほとんどを占めていた

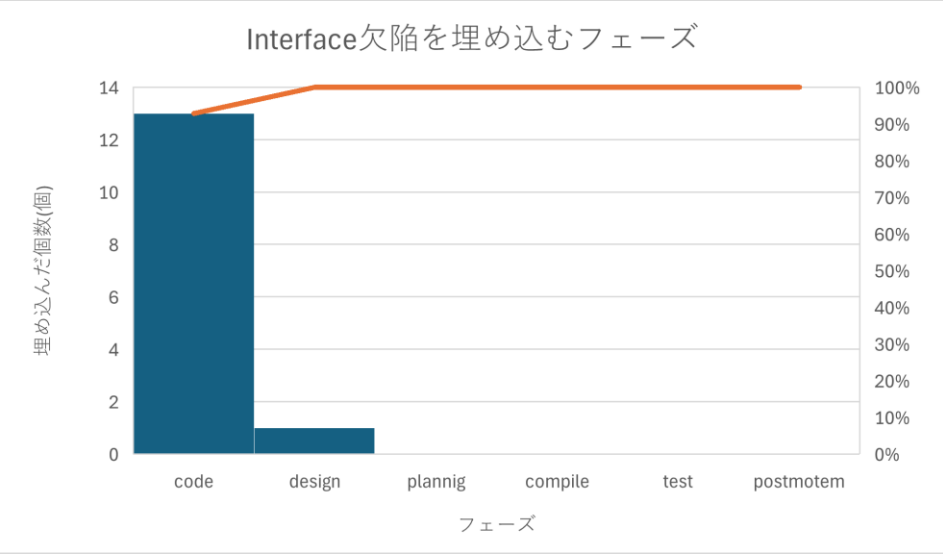
3.4-1 各欠陥型が埋め込まれるフェーズ





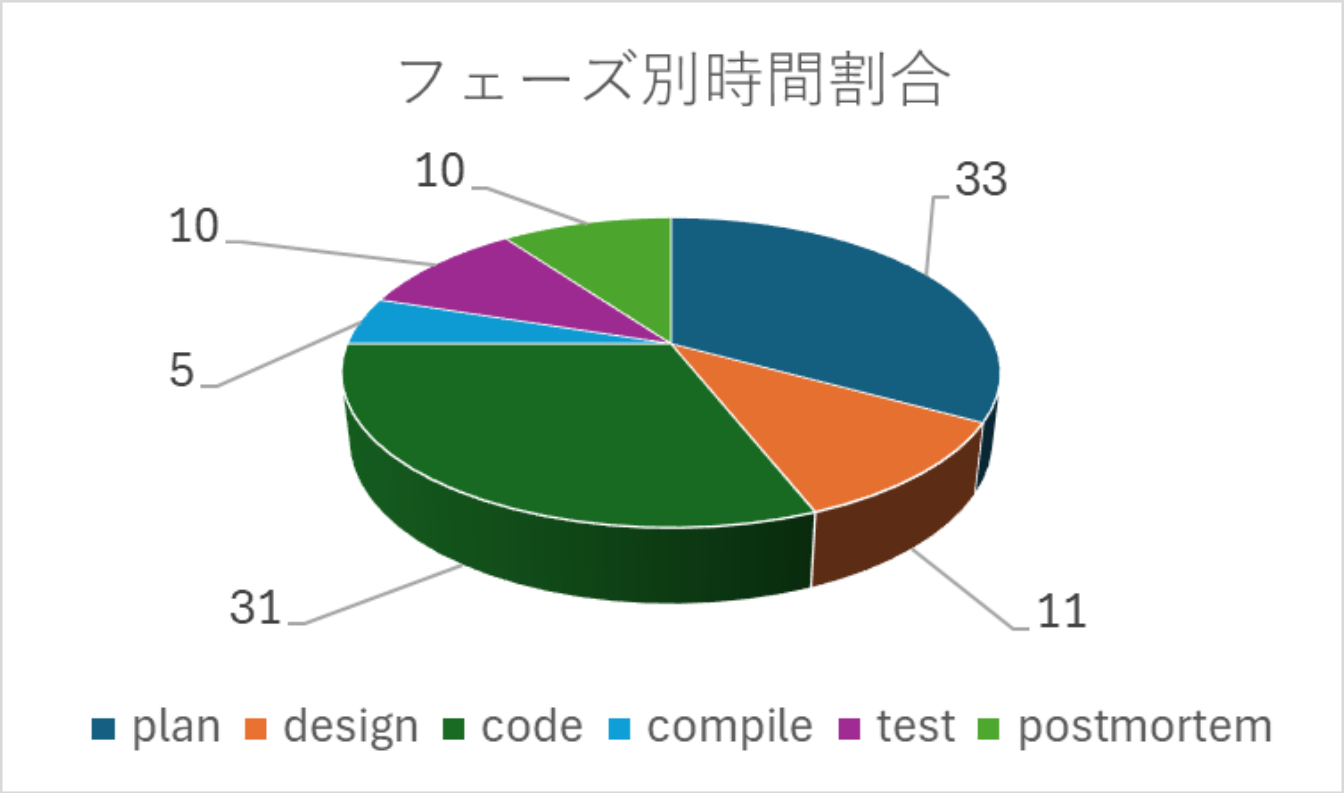
3.4-2 各欠陥型が埋め込まれるフェーズ





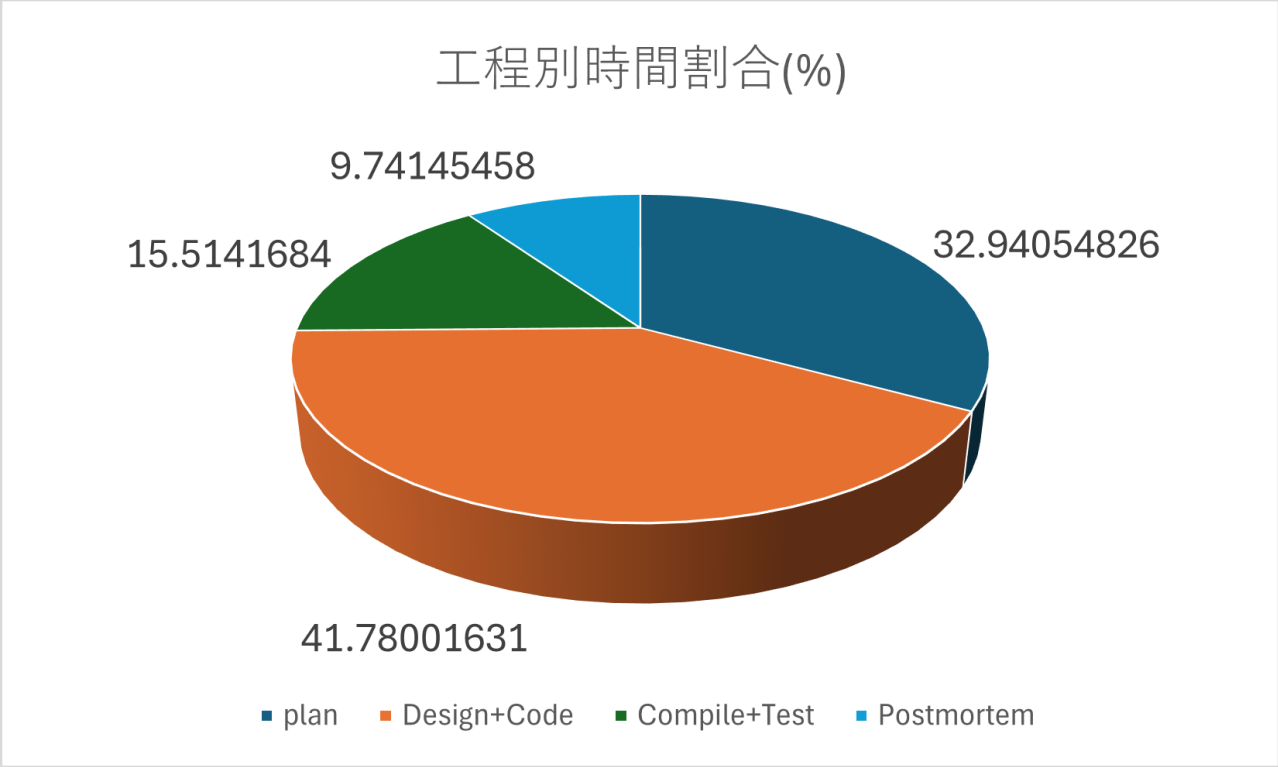
図にしていないがdocumentationはすべてplanで埋め込まれたdocumentationを除くすべての欠陥型でcodeで最も多く埋め込まれ、次にdesignと続いた。

4.1 各フェーズの時間割合(%)



全6工程のうち3分の1の時間をcodeが占めている。
設計が甘くcodeでロジックを考えなおす時間があることが問題である

4.2 各フェーズ時間割合(%)



この視点からだと、予防的活動/是正的活動>2となっており
比較的前工程で作業の多くを終わらせられているともとれる

フェーズ時間割合に対する考察

4.1,4.2より、codeに時間をかけることで、是正的活動にかける時間の割合が低くなっている。

しかし、codeでロジックを考えている時間があるのでcodeに時間をかけすぎるのではなくdesignで実装が一意に定まるように詳細に設計する必要がある。

4.3 1000時間書くとしたら?

フェーズ	割合(%)	X=1000時間のうち何時間?(h)
Design	10.643	106.43
Code	31.149	311.49
Test	10.343	103.43

計算式:割合×10

4.4 1000時間書いた時のLOCと欠陥数

生産性：13.38(LOC/h)

テストの欠陥密度： 43.1(個/KLOC)

より、

13.38×1000=13380(LOC)

43.1×13.38=576.678(個)

まとめ

見積もり精度の向上: ベースコード利用時も、追加・修正部分の規模を「プロキシの規模範囲」で詳細に見積もり、計画の精度を高める。

設計フェーズの強化: designで実装を一意に定める詳細設計を徹底し、codeでのロジック検討時間や欠陥埋め込みを削減する。

欠陥除去の早期化: designでのレビュー強化と、codeでの自己チェック徹底により、特にsyntax欠陥の予防的活動での早期発見・除去を促進する。